

Library Management System

Softwareentwicklung 3, SoSe 2023



BookSTACK

Shaqaiq, Adib as448@hdm-stuttgart.de
Öhler, Jonas jo041@hdm-stuttgart.de

Projektdokumentation

Run BookStack

Für das Projekt wird OpenJDK 19 benötigt. Für die Datenbank muss nichts weiter getan werden, da wir mit einer Embedded Datenbank gearbeitet haben.

Um das Programm zu starten, muss der JavaFX ausgeführt werden und nicht der ganze Git Ordner (se3-project).

Anforderungsanalyse

Rudimentäre Anforderungen

Benutzerverwaltung: Unser System sollte in der Lage sein, Informationen über Bibliotheks-Mitglieder zu speichern. Dazu zählen Vor - und Nachname, Username und das Passwort. Es sollte auch die Möglichkeit bieten, Benutzerkonten anzulegen, zu aktualisieren und zu löschen.

Medienverwaltung: Es soll möglich sein, Informationen über die verfügbaren Medien in der Bibliothek zu speichern und zu verwalten. Dazu gehören Bücher, Filme und bei Bedarf noch weitere Medien. Über Textfelder, sollen Informationen wie ISBN, Titel, Autor und Verfügbarkeitsstatus erfasst und in einer Datenbank gespeichert werden. Des Weiteren sollen Funktionen zur Aufnahme neuer Medien, Aktualisierung von Informationen und Entfernung von veralteten oder beschädigten Medien implementiert werden.

Ausleih- und Rückgabe Verwaltung: Das LMS sollte die Verwaltung von Ausleih- und Rückgabe Vorgängen unterstützen. Bei der Rückgabe eines Mediums sollte das System den Verfügbarkeitsstatus aktualisieren und eventuell Gebühren berechnen, wenn das Medium verspätet zurückgegeben wurde.

Benutzerfreundliche Oberfläche: Wir wollen unsere GUI so gestalten, dass eine einfache Navigation durch das System möglich ist. Benutzer sollten in der Lage sein, sich leicht anzumelden und auf die verschiedenen Funktionen des LMS zuzugreifen. Die Darstellung der Informationen sollte übersichtlich sein, mit klaren Beschriftungen und Symbolen.

Kategorisierung der Anforderungen

Funktionale Anforderungen:

- Wenn ein Medium ausgeliehen wird, wird dem Medium eine User - ID zugeordnet
- Beim Ausleihen wird ein Eintrag angelegt, der das Ausleih- und Rückgabe Datum der Transaktion beinhaltet.
- Nach der Rückgabe eines Mediums wird die entsprechende Transaktion als abgeschlossen markiert.
- Ausgeliehene Medien werden nicht beim Ausleihen angezeigt.
- Beim Zurückgeben eines Mediums werden nur die ausgeliehenen Medien des ausgewählten Users angezeigt.

Nicht-Funktionale Anforderungen:

- Es soll eine einfache Navigation durch das System ermöglicht werden.
- Die Anmeldung und der Zugriff auf die verschiedenen Funktionen sollten leicht sein.
- Die Darstellung der Informationen sollte übersichtlich sein, mit klaren Beschriftungen und Symbolen.

Fachliche Anforderungen:

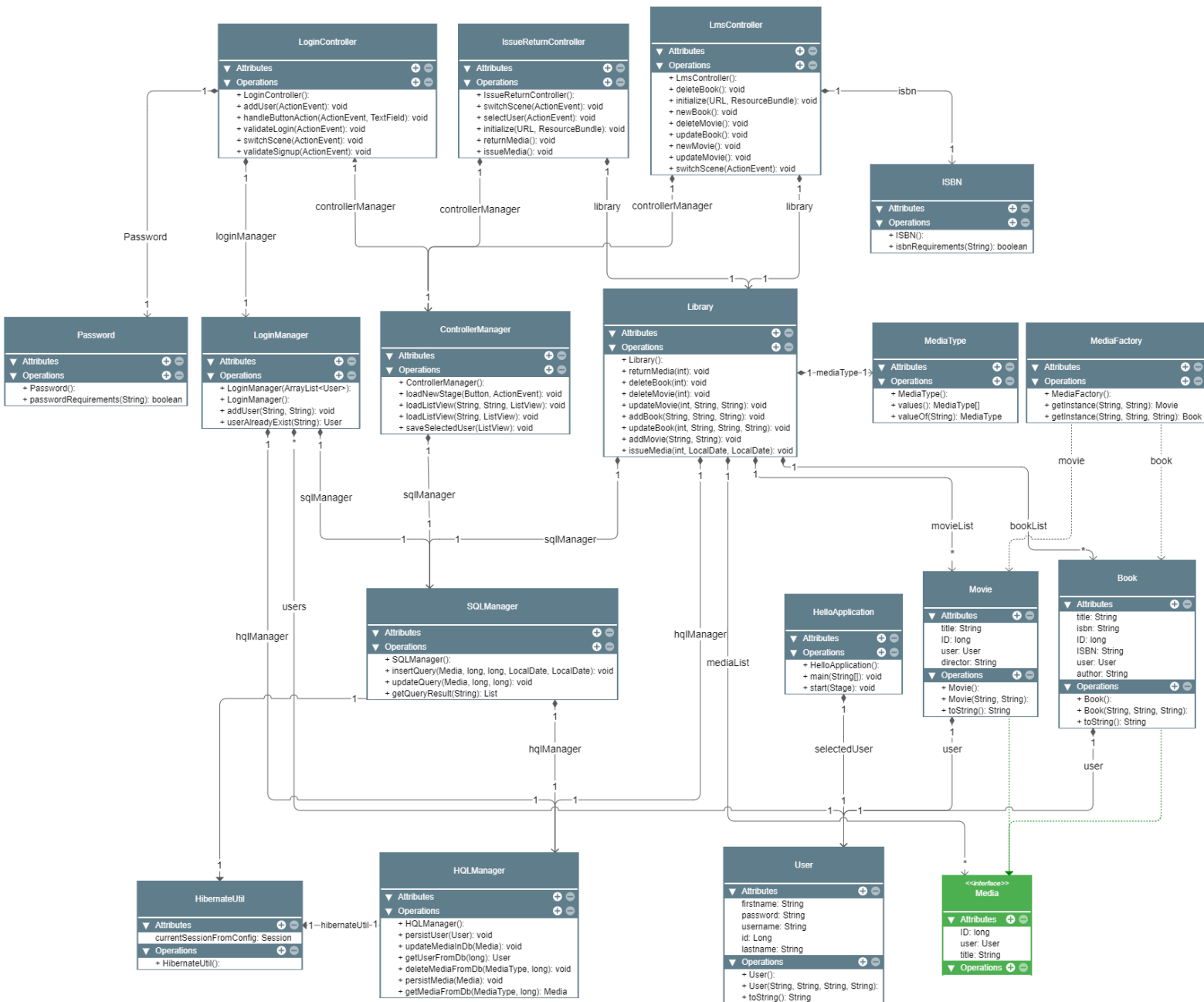
- Beim Anlegen eines neuen Buches wird die ISBN auf Korrektheit überprüft
Commit: 352f3b06, 9caf2e59
- Beim Anlegen eines neuen Users soll das Passwort auf seine formale Korrektheit überprüft werden.
Commit: bf1a369d, bb786996
- Medien können nicht erstellt und aktualisiert werden, wenn nicht alle Informationen übergeben werden.
Commit: 2b1e57a1
- Passwort muss zweimal eingegeben werden, dabei müssen beide Passwörter übereinstimmen.
Commit: bf1a369d, bb786996
- Wenn ein Username bereits existiert, kann er nicht erneut angelegt werden.
Commit: bf1a369d, bb786996
- Das Rückgabedatum darf nicht vor dem Ausleihdatum gesetzt werden.
Commit: 08971a9e

Technische Anforderungen

- Es wird rechnerisch überprüft, ob die ISBN valide ist.
Commit: 352f3b06, 9caf2e59
- Es wird mit Checks geprüft, ob das Passwort bestimmte Kriterien erfüllt.
Commit: bf1a369d, bb786996
- Sollte beim Erstellen bzw aktualisieren von Medien Felder nicht ausgefüllt worden sein, soll eine entsprechende Fehlermeldung erscheinen.
Commit: 2b1e57a1
- Die Anwendung soll über eine Datenbank ihre Daten beziehen.
Commit: b648abc2, 105a1657
- Die UI wird über fxm1 und CSS generiert.
Commit: 41ac81d3, eff9a68e
- Beim Registrieren werden beide Passwörter mit Checks überprüft. Sollten sie nicht gleich sein, erscheint eine entsprechende Fehlermeldung
Commit: bf1a369d, bb786996
- Sollte beim Erstellen eines Users der Username bereits vorhanden sein, soll eine entsprechende Fehlermeldung erscheinen.
Commit: bf1a369d, bb786996
- Es wird ein Check gemacht, ob das Ausleihdatum vor dem Rückgabedatum gesetzt wurde.
Commit: 08971a9e

Softwarearchitektur & Clean Code

Klassendiagramm:



Architekturmuster

Wir haben uns für eine schichtbasierte Architektur entschieden. Dies ermöglicht eine klare Trennung von Verantwortlichkeiten und Aufgaben. Dadurch können verschiedene Schichten spezifische Funktionen übernehmen und der Code wird besser organisiert und wartbarer. Zum Beispiel können separate Schichten für die Benutzerschnittstelle, Geschäftslogik und Datenbankzugriff verwendet werden. Änderungen in einer Schicht haben keinen Einfluss auf andere Teile des Systems, da sie isoliert bleiben. Jede Schicht kann unabhängig entwickelt und getestet werden.

SOLID - Prinzipien

1. Single Responsibility Principle (SRP):
Jede Klasse sollte nur eine bestimmte Aufgabe erfüllen.
Die Aufgabe unserer Library-Klasse beschränkt sich ausschließlich auf die Verwaltung der Medien.
2. Open-Closed Principle (OCP):
Jede Klassen sollte offen für Erweiterungen sein, aber geschlossen für Veränderungen.
Durch die Verwendung unseres Media-Interfaces können neue Medientypen hinzugefügt werden, ohne dass Änderungen am vorhandenen Code erforderlich sind.
3. Liskov Substitution Principle (LSP):
Unterklassen erweitern Oberklassen. Unterklassen dürfen das Verhalten der Oberklassen nicht einschränken.
In unserem Library Management System enthalten alle Medienklassen dieselben Methoden.
4. Interface Segregation Principle (ISP):
Große Schnittstellen in kleinere, modulare Schnittstellen aufteilen.
In unserem LMS verwenden wir nur ein Interface, wobei wir jedoch darauf geachtet haben, unnötige Abhängigkeiten zu vermeiden, indem wir nur die notwendigsten Methoden in das Interface implementiert haben.
5. Dependency Inversion Principle (DIP)
Wir haben darauf geachtet, konkrete Implementierungen zu vermeiden, indem wir über das Media Interface auf unsere implementierenden Klassen zugreifen. Dies geschieht beispielsweise in unserer HQLManager Klasse, bei der alle CRUD Operationen über das Media Interface durchgeführt werden.

Kommunikation zwischen den Komponenten

Die wichtigsten Komponenten des Systems sind die Benutzeroberfläche, die Datenbank und die Geschäftslogik.

Die Benutzeroberfläche ermöglicht es den Bibliotheksmitarbeitern auf das System zuzugreifen und verschiedene Aktionen durchzuführen, wie z.B. das Ausleihen oder Zurückgeben von Büchern. Die Benutzeroberfläche sendet Anfragen an die Geschäftslogik, um diese Aktionen auszuführen.

Die Geschäftslogik ist die Hauptkomponente des Systems und kümmert sich um die Verarbeitung der Anfragen der Benutzeroberfläche. Sie nimmt die Anfragen entgegen, verarbeitet sie und greift bei Bedarf auf die Datenbank zu. Die Geschäftslogik stellt sicher, dass alle Anfragen den vorgegebenen Regeln entsprechen und prüft die Integrität der Daten. Die Datenbank des Library Management Systems speichert Informationen über Medien, Benutzer und Transaktionen. Die Geschäftslogik verwendet die Datenbank, um Daten abzurufen oder zu aktualisieren.

Architektonische Entscheidungen

1. Trennung der Verantwortlichkeiten:
Jede Schicht hat eine spezifische Aufgabe und ist für einen bestimmten Abschnitt des Systems zuständig, z. B. Datenzugriff, Geschäftslogik oder Benutzeroberfläche. Dies erleichtert das Verständnis und Wartung des Systems.
2. Modularität und Wiederverwendbarkeit:
Durch die Aufteilung in Schichten können die einzelnen Teile unabhängig voneinander entwickelt und gewartet werden. Dadurch können Komponenten leicht wiederverwendet werden und es ist einfacher, neue Funktionen hinzuzufügen. Änderungen in einer Schicht beeinflussen andere Schichten nicht, solange die Schnittstellen zwischen ihnen unverändert bleiben.
3. Flexibilität und Erweiterbarkeit:
Es können neue Funktionen zur bestehenden Architektur hinzugefügt werden. Das System sollte flexibel sein, um sich verändernden Anforderungen anpassen zu können.
4. Testbarkeit:
Durch die Trennung der Verantwortlichkeiten in den Schichten wird das Testen des Systems erleichtert. Jede Schicht kann unabhängig getestet werden, was die Fehlersuche und das Durchführen von Tests vereinfacht.

Architecture Decision Record: Relationale Datenbank

Verwendung einer relationalen Datenbank im Gegensatz zu einer nicht relationalen Datenbank für ein Library Management System

Kontext- und Problemstellung

Wir müssen entscheiden, welche Datenbank wir für unser Library Management System verwenden sollen. Die Datenbank ist wichtig für die Speicherung, Organisation und Abfrage von Bibliotheksdaten. Wir haben verschiedene Optionen in Betracht gezogen, einschließlich relationaler und nicht relationaler Datenbanken, aber wir denken, dass eine relationale Datenbank am besten für unser System geeignet wäre.

Entscheidung

Wir haben uns dafür entschieden, eine relationale Datenbank anstelle einer nicht relationalen Datenbank für unser Library Management System zu nutzen.

Unsere Entscheidung beruht auf folgenden Gründen:

1. Strukturierte Datenorganisation:
Relationale Datenbanken bieten klare Beziehungen und Constraints zwischen den Daten, was eine bessere Modellierung und den Zugriff auf komplexe Beziehungen und Abfragen ermöglicht. Nicht relationale Datenbanken haben eine flexible Struktur und sind für unstrukturierte Daten geeignet, aber möglicherweise nicht für unsere Systemanforderungen.
2. Datenintegrität und Konsistenz:
Relationale Datenbanken haben Mechanismen für Datenintegrität und Konsistenz, wie Primärschlüssel, Fremdschlüssel und eindeutige Werte. Dadurch werden die Daten im Library Management System korrekt und konsistent gespeichert. Im Gegensatz dazu haben nicht relationale Datenbanken weniger strenge Konsistenz-Mechanismen und können inkonsistente Daten zulassen, was die Zuverlässigkeit und Genauigkeit der Daten beeinträchtigen kann.
3. Komplexe Abfragen und Analysen:
Ein Library Management System kann komplexe Abfragen und Analysen erfordern, um Informationen wie Transaktionen, Benutzer/-Medieninformationen zu filtern. Nicht relationale Datenbanken können Einschränkungen bei komplexen Abfragen und Joins haben, da sie nicht für umfangreiche Datenbeziehungen optimiert sind.

Konsequenzen

Die Entscheidung, eine relationale Datenbank im Gegensatz zu einer nicht relationalen Datenbank für unser Library Management System zu verwenden, hat folgende Konsequenzen:

- Strukturierte Datenmodellierung: Die Verwendung einer relationalen Datenbank erfordert eine genaue Modellierung der Daten und die Definition von Tabellen, Beziehungen und Constraints. Dies kann einen zusätzlichen Aufwand bei der Entwicklung und Wartung des Systems bedeuten.
- Abhängigkeit von SQL: Relationale Datenbanken nutzen die SQL-Abfragesprache, was bedeutet, dass Entwickler über Erfahrung mit SQL verfügen müssen, um effektiv mit der Datenbank zu arbeiten. Es kann erforderlich sein, die Entwickler entsprechend zu schulen.
- Skalierbarkeit: Relationale Datenbanken sind gut für Systeme geeignet, bei denen komplexe Abfragen und Transaktionen im Vordergrund stehen. Allerdings können sie bei sehr großen Datenmengen überlastet werden.

Architecture Decision Record: H2-Datenbank

Auswahl von H2 als Datenbankverwaltungssystem für ein Library Management System

Kontext- und Problemstellung

Wir stehen vor der Entscheidung, welche Datenbanktechnologie wir für unser Library Management System verwenden sollen. Das Library Management System ist eine Anwendung, die die Verwaltung und Organisation von Büchern, Medien und Ausleihvorgängen in einer Bibliothek unterstützt. Wir haben verschiedene Optionen in Betracht gezogen und sind zu dem Schluss gekommen, dass die Verwendung einer H2-Datenbank für unser System von Vorteil sein könnte.

Entscheidung

Wir haben uns dafür entschieden, eine H2-Datenbank in unserem Library Management System zu nutzen. Unsere Entscheidung beruht auf folgenden Gründen:

1. Einfache Einrichtung und Verwaltung: H2 ist eine leichtgewichtige und in Java geschriebene relationale Datenbank. Die Einrichtung und Verwaltung der H2-Datenbank ist einfach und erfordert keine komplexen Konfigurationsschritte. Dies ermöglicht es uns, schnell mit der Entwicklung unserer Anwendung zu beginnen.
2. Gute Performance: H2 hat eine hohe Leistungsfähigkeit und Geschwindigkeit. Es kann große Datenmengen effizient verarbeiten und bietet gleichzeitig eine schnelle Antwortzeit für Abfragen. Da unser Library Management System umfangreiche Informationen enthält, ist es wichtig, dass die Datenbanktechnologie eine schnelle Antwortzeit für die Benutzer ermöglicht.
3. Plattformunabhängigkeit: H2-Datenbanken sind plattformunabhängig und können auf verschiedenen Betriebssystemen ausgeführt werden. Dies ermöglicht uns, unsere Anwendung auf verschiedenen Geräten zu betreiben, was Flexibilität und Skalierbarkeit gewährleistet.

Konsequenzen

- Unser Entwicklungsteam muss sich mit der H2-Datenbank vertraut machen und über die erforderlichen Kenntnisse und Fähigkeiten verfügen, um effektiv damit zu arbeiten.

Architecture Decision Record: Programmiersprache Java

Verwendung von Java als Programmiersprache für ein Library Management System

Kontext- und Problemstellung

Wir stehen vor der Entscheidung, welche Programmiersprache wir für die Entwicklung unseres Library Management Systems verwenden sollen. Das Library Management System ist eine Anwendung, die die Verwaltung und Organisation von Büchern, Medien und Ausleihvorgängen in einer Bibliothek unterstützt. Wir haben verschiedene Optionen in Betracht gezogen und sind zu dem Schluss gekommen, dass die Verwendung von Java als Programmiersprache für unser System von Vorteil sein könnte.

Entscheidung

Unsere Entscheidung beruht auf folgenden Gründen:

1. Plattformunabhängigkeit:
Java ist eine plattformunabhängige Programmiersprache. Das bedeutet, dass das entwickelte Library Management System auf verschiedenen Betriebssystemen wie Windows, macOS und Linux ausgeführt werden kann, ohne dass größere Änderungen erforderlich sind. Dies ermöglicht eine Nutzung auf verschiedenen Systemen.
2. Umfangreiche Bibliotheken und Frameworks:
Java bietet eine Vielzahl von Bibliotheken und Frameworks, die speziell für die Entwicklung von Enterprise-Anwendungen entwickelt wurden. Diese Bibliotheken und Frameworks vereinfachen die Implementierung gängiger Funktionen, wie beispielsweise die Anbindung an Datenbanken, die Verarbeitung von Daten und die Benutzerschnittstelle. Dadurch können wir die Entwicklungszeit verkürzen.
3. Objektorientierte Programmierung:
Java ist eine objektorientierte Programmiersprache, die die Entwicklung von modularen und wiederverwendbaren Code ermöglicht. In einem Library Management System, das verschiedene Komponenten wie Bücher, Ausleihen, Benutzer Profile usw. umfasst, bietet eine OO Sprache den Vorteil der Strukturierung des Codes und der leichten Erweiterbarkeit.
4. Sicherheit und Stabilität:
Java legt großen Wert auf Sicherheit und Stabilität. Die Sprache und die zugehörige Laufzeitumgebung bieten Mechanismen, um potenzielle Sicherheitslücken zu vermeiden. Ein Beispiel hierfür sind die Access Modifier in Java.

Konsequenzen

- Wir brauchen Entwickler, die Erfahrung mit Java haben.
- Jeder in der Sprache geschriebene Code muss mit der Java Virtual Machine (JVM) kompatibel sein.
- Die Integration neuer Features, Funktionalitäten oder Frameworks kann einige Herausforderungen mit sich bringen.
- Die Entwicklung großer, komplexer Projekte in Java erfordert möglicherweise mehr Zeit und Ressourcen.

GIT

Wir haben versucht, uns an den Workflow des Gitflows zu halten. Dabei haben wir mit einem Master-Branch gearbeitet. Der Master-Branch repräsentiert den stabilen Hauptzweig unseres Projekts. Er enthält nur stabile, vollständig getestete und freigegebene Funktionen. Der Developer-Branch ist der Hauptentwicklungsbranch. Er enthält den aktuellen Entwicklungsstand unseres Projekts. Neue Features und Änderungen wurden in diesem Branch entwickelt und getestet, bevor sie in den Master-Branch gepusht wurden.

Für jedes neue Feature oder jede neue Funktion haben wir einen Feature-Branch erstellt.

Diese Branches haben wir vom Developer Branch abgezweigt, um neue, vom Developer Branch unabhängige Features zu entwickeln. Sobald das Feature fertiggestellt war, wurde der Branch in den Develop-Branch gemerged.

Bei den Commits haben wir darauf geachtet, dass wir Änderungen mit "change/update/modify", hinzufügen von neuen Features mit "add" sowie löschen von code bestandteilen mit "delete" benennen um somit einen guten Workflow zu gewährleisten.

Reflexion

Was habe ich gelernt?

Adib: Ich habe während des Projekts gelernt, wie man mit Git Workflows arbeitet und JPA für ORM einsetzt. Außerdem habe ich ein tieferes Verständnis für die Verwendung von Clean Code Prinzipien und Entscheidungen bezüglich der Softwarearchitektur gewonnen.

Jonas: Im Verlauf der Entwicklung habe ich gelernt, mich an ein einheitliches Naming bei Git Commits zu halten und dadurch einen guten Workflow zu gewährleisten. Für mich war es neu, mit Feature Branches zu arbeiten, um somit gleichzeitig, unabhängig von anderen Entwicklern, an verschiedenen Funktionen oder Features unserer Software zu arbeiten. Außerdem habe ich durch die Nutzung von Hibernate neue Erkenntnisse darüber erlangt, wie man mit Hilfe von Frameworks effizienter programmieren kann.

Wo habe ich noch Lücken?

Adib: Ich habe noch Lücken beim Thema Containern, da wir in unserem Projekt nicht mit Containern gearbeitet haben und ich mich nicht ausführlich mit dem Thema auseinandergesetzt habe.

Jonas: Ich habe noch Lücken beim Thema CI/CD, da wir in unserem Projekt keinen lauffähigen Build-Lifecycle implementiert haben und wir uns nicht ausreichend genug mit dem Thema beschäftigt haben.

Bei diesen Themen bin ich über meinen Schatten gesprungen:

Adib: Das Thema Anforderungsanalyse hat mich sehr viel Überwindung gekostet, da ich meistens erst während der Entwicklung mit den Anforderungen eines Softwareprojekts beschäftige und nicht bereits vorab.

Jonas: Mich hat das Thema Clean Code herausgefordert, da ich teilweise immer noch redundanten Code schreibe anstatt ihn sinnvoll zusammenzufassen.